



MANUAL DE USUARIO

RoboLab MDE

Guía para docentes, estudiantes y entusiastas de la robótica

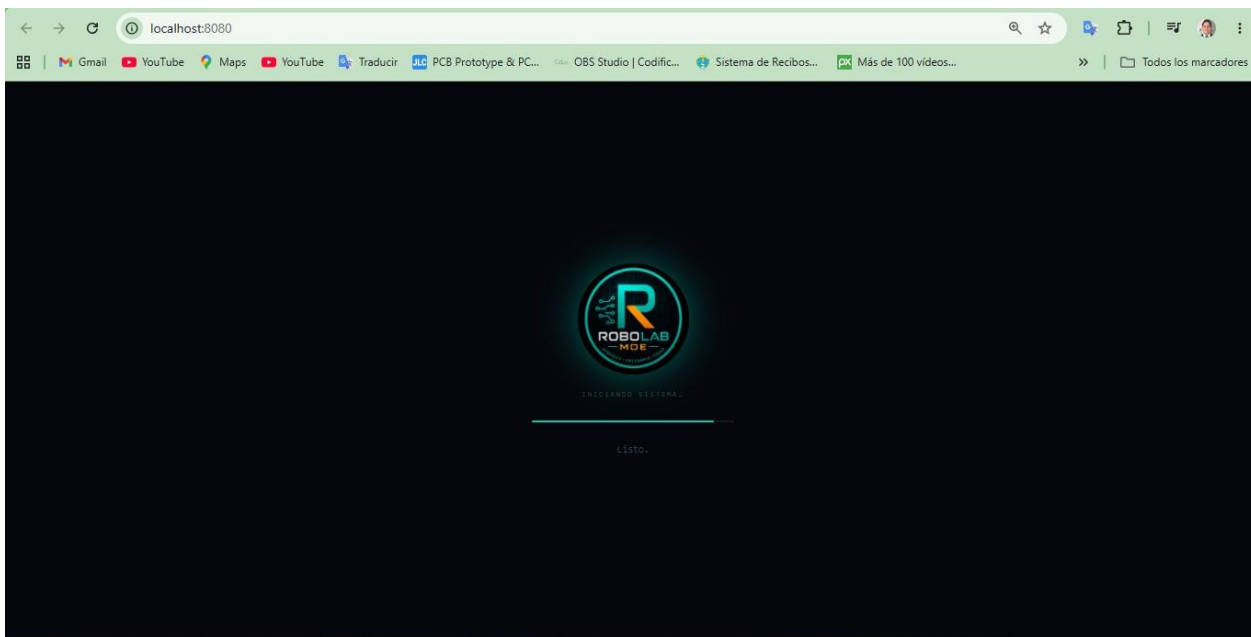
APRENDER · PROGRAMAR · CREAR

Presentación

RoboLab es una plataforma educativa interactiva diseñada para aprender robótica, electrónica y programación de forma progresiva y práctica. Funciona completamente en el navegador web, sin necesidad de instalar software adicional, y fue pensada para que cualquier persona — con o sin experiencia previa — pueda comenzar a programar robots desde el primer día.

Fue creado para resolver un problema concreto en la educación tecnológica: la brecha entre la teoría y la práctica. RoboLab integra simulación, teoría, desafíos y retroalimentación en un único entorno, haciendo que el aprendizaje sea inmediato, visible y significativo.

2



Pantalla de inicio de RoboLab — carga en pocos segundos desde el navegador

¿Para quién es RoboLab?

Perfil	¿Cómo usa RoboLab?
☐ Docentes	Planificar clases, talleres y evaluaciones. Acceder a la Vista Docente para monitorear progreso.
☐ Estudiantes	Avanzar de forma autónoma por los desafíos y unidades curriculares.
☐ Talleristas	Conducir actividades de robótica en contextos no formales o extracurriculares.
☐ Makers y aficionados	Explorar electrónica y programación de forma autodidacta sin requisitos previos.
☐ Sin experiencia previa	RoboLab está diseñado para comenzar desde cero absoluto.

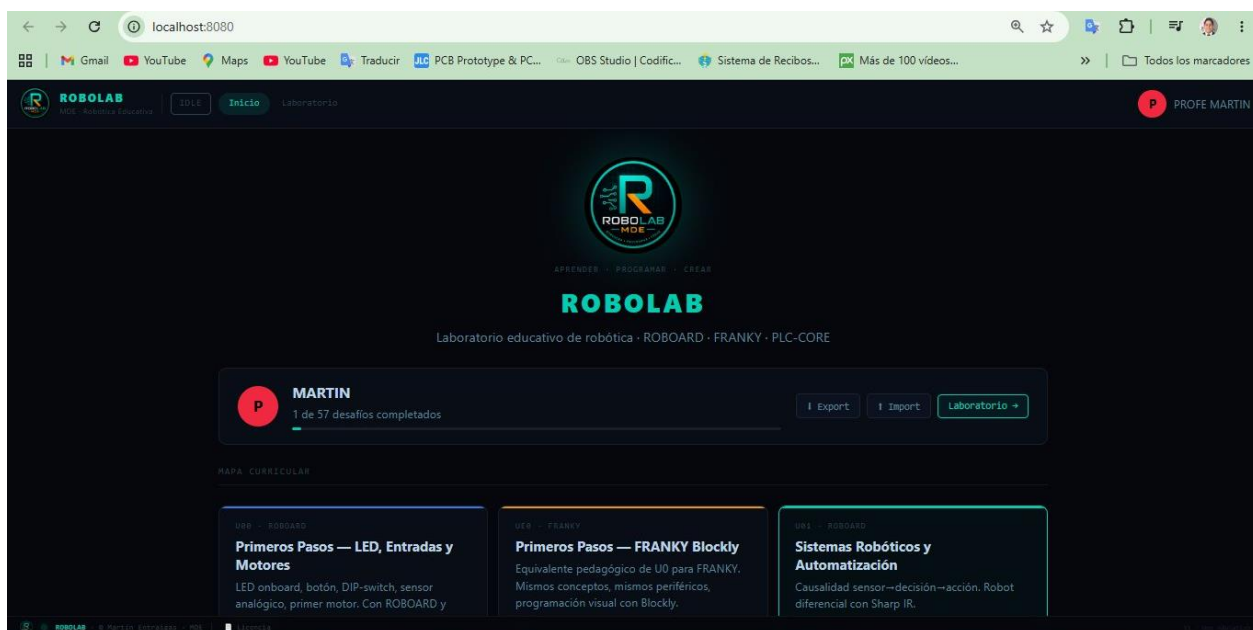
Capítulo 1 — Conociendo RoboLab

¿Qué es RoboLab?

RoboLab es un entorno integral de aprendizaje para robótica educativa. No es simplemente una colección de ejercicios: es un laboratorio virtual donde el estudiante experimenta, programa y observa los resultados en tiempo real sobre un simulador de robot.

Compatible con dos plataformas de hardware real — ROBOARD 6.0 y FRANKY 4.0 — RoboLab permite trabajar en modo simulado o conectar el robot real para subir el código y observar el comportamiento físico.

3



Pantalla principal de RoboLab — mapa curricular con todas las unidades disponibles

Filosofía: Aprender Haciendo

La base pedagógica de RoboLab es el aprendizaje activo. En lugar de leer teoría y después intentar aplicarla, el estudiante enfrenta un desafío concreto desde el primer momento: recibe un código incompleto, debe completarlo, ejecutarlo y verificar que el robot responde correctamente en el simulador.

- Aprender haciendo — la acción precede a la teoría
- Experimentación — probar, fallar y corregir es parte del proceso
- Resolución de problemas — cada desafío plantea una situación real
- Desarrollo de proyectos — los conceptos se integran en aplicaciones concretas
- Progresión gradual — la complejidad aumenta de forma controlada
- Construcción de conocimientos significativos — el estudiante comprende, no memoriza

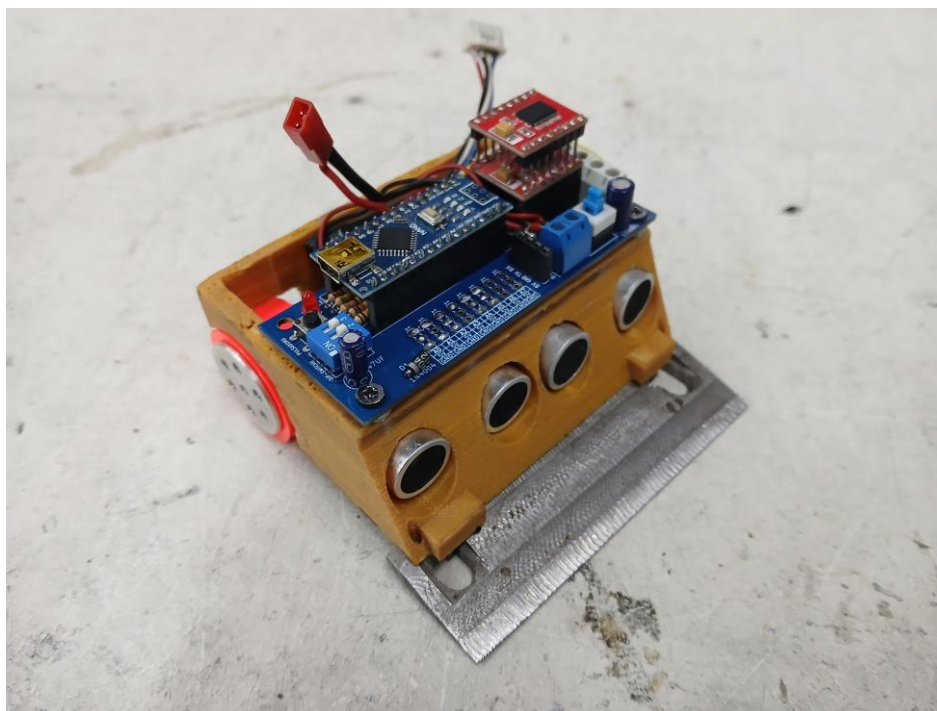


Robots contruidos con las plataformas ROBOARD 6.0 en arena de competencia

Las Plataformas de Hardware Real

RoboLab simula fielmente dos plataformas físicas reales. Cuando el estudiante esté listo para dar el salto al hardware, el código que programó en el simulador funciona directamente en el robot real.

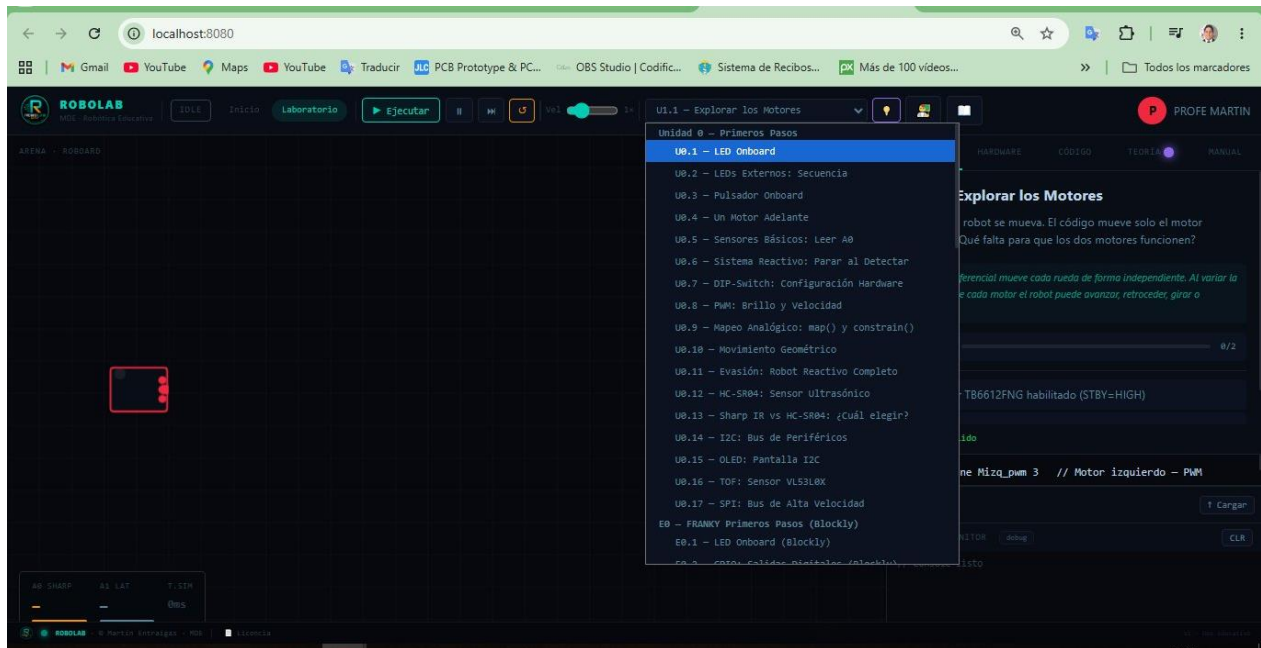
 FRANKY 4.0 PLATAFORMA EDUCATIVA PARA ROBÓTICA REAL	 ROBOARD 6.0 PLATAFORMA EDUCATIVA PARA ROBÓTICA DE COMPETENCIA
FRANKY 4.0 — Plataforma educativa para robótica real	ROBOARD 6.0 — Plataforma educativa para robótica de competencia



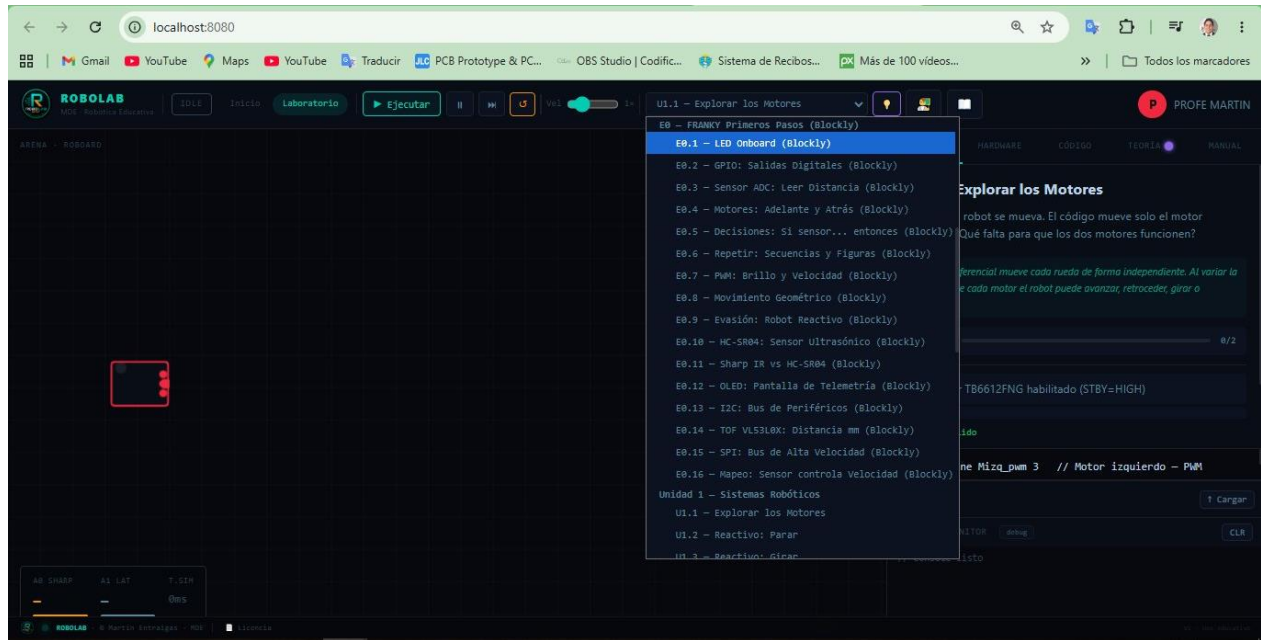
Robot ROBOARD 6.0 — estructura impresa en 3D con electrónica expuesta para aprendizaje

Capítulo 2 — Organización de RoboLab

RoboLab organiza el aprendizaje en unidades curriculares progresivas. Cada unidad agrupa desafíos relacionados con un tema específico. La progresión está diseñada para que el estudiante construya sobre lo aprendido en las unidades anteriores.



Menú de desafíos — Unidad 0 (ROBOARD): 17 desafíos en C++/Arduino



Menú de desafíos — Unidad E0 (FRANKY Blockly): 16 desafíos con programación visual

Tabla Resumen de Unidades

Código	Nombre	Desafíos	Contenido principal
--------	--------	----------	---------------------

U0	Primeros Pasos (ROBOARD)	17	Introducción en C++/Arduino: LED, botones, motores, sensores, I2C, SPI.
E0	Primeros Pasos (FRANKY/Blockly)	16	Equivalente a U0 con programación visual por bloques para FRANKY 4.0.
U1	Sistemas Robóticos	3	Robot diferencial reactivo. Causalidad sensor → decisión → acción.
U2	Programación Interactiva	3	Funciones, millis(), control proporcional. Código estructurado.
U3	Sensado y Datos	6+	Sensores IR, minisumo autónomo, seguidor de línea, control PD.
E1–E5	FRANKY Etapas Avanzadas	8	Control remoto, telemetría, modos autónomos, minisumo PD.

Unidad U0 — Primeros Pasos con ROBOARD (C++/Arduino)

Es la unidad de entrada para quienes trabajan con ROBOARD 6.0. Cubre los fundamentos del hardware y la programación. Cada desafío introduce un concepto nuevo sobre el anterior.

ID	Desafío	Qué aprende el estudiante
U0.1	LED Onboard	Encender y apagar el LED integrado con digitalWrite().
U0.2	LEDs Externos: Secuencia	Controlar múltiples LEDs como salidas digitales.
U0.3	Pulsador Onboard	Leer el estado de un botón con digitalRead().
U0.4	Un Motor Adelante	Mover el primer motor del robot.
U0.5	Sensores Básicos: Leer A0	Leer valores analógicos con analogRead().
U0.6	Sistema Reactivo: Parar al Detectar	El robot reacciona al entorno deteniendo sus motores.
U0.7	DIP-Switch: Configuración Hardware	Configurar el robot mediante interruptores físicos.
U0.8	PWM: Brillo y Velocidad	Control de velocidad/brillo con señal PWM.
U0.9	Mapeo Analógico: map() y constrain()	Transformar rangos de valores analógicos.
U0.10	Movimiento Geométrico	Programar trayectorias: cuadrados, curvas, figuras.
U0.11	Evasión: Robot Reactivo Completo	Robot que evita obstáculos de forma autónoma.
U0.12	HC-SR04: Sensor Ultrasónico	Medir distancias con sonar ultrasónico.
U0.13	Sharp IR vs HC-SR04: ¿Cuál elegir?	Comparar y elegir entre tipos de sensores de distancia.
U0.14	I2C: Bus de Periféricos	Comunicar dispositivos por bus I2C.

U0.15	OLED: Pantalla I2C	Mostrar datos en pantalla OLED.
U0.16	TOF: Sensor VL53L0X	Sensor de distancia por tiempo de vuelo.
U0.17	SPI: Bus de Alta Velocidad	Comunicación de alta velocidad por bus SPI.

Unidad E0 — Primeros Pasos con FRANKY (Programación Visual Blockly)

8

E0 es el equivalente pedagógico de U0 para la plataforma FRANKY 4.0. Usa programación visual con bloques (Blockly), ideal para introducir la robótica sin necesidad de escribir código en texto.

ID	Desafío	Qué aprende
E0.1	LED Onboard (Blockly)	Encender el LED con bloques visuales.
E0.2	GPIO: Salidas Digitales (Blockly)	Controlar salidas con bloques.
E0.3	Sensor ADC: Leer Distancia (Blockly)	Leer distancias con bloques.
E0.4	Motores: Adelante y Atrás (Blockly)	Mover el robot con bloques de motor.
E0.5	Decisiones: Si sensor... entonces (Blockly)	Estructura condicional con bloques.
E0.6	Repetir: Secuencias y Figuras (Blockly)	Bucles y repetición visual.
E0.7	PWM: Brillo y Velocidad (Blockly)	Control de velocidad con bloques PWM.
E0.8	Movimiento Geométrico (Blockly)	Figuras y trayectorias con bloques.
E0.9	Evasión: Robot Reactivo (Blockly)	Robot que evade obstáculos con bloques.
E0.10	HC-SR04: Sensor Ultrasónico (Blockly)	Sensor ultrasónico con bloques.
E0.11	Sharp IR vs HC-SR04 (Blockly)	Comparar sensores con bloques.
E0.12	OLED: Pantalla de Telemetría (Blockly)	Mostrar datos en OLED con bloques.
E0.13	I2C: Bus de Periféricos (Blockly)	Bus I2C con bloques visuales.
E0.14	TOF VL53L0X: Distancia mm (Blockly)	Sensor TOF con bloques.
E0.15	SPI: Bus de Alta Velocidad (Blockly)	Bus SPI con bloques.
E0.16	Mapeo: Sensor controla Velocidad (Blockly)	Mapeo analógico para controlar velocidad.

Unidades 1 a 3 — Progresión Avanzada con ROBOARD



Unidad 1 — Sistemas Robóticos (3 desafíos)

El estudiante descubre la causalidad sensor → decisión → acción en un robot diferencial. U1.1 Explorar los Motores · U1.2 Reactivo: Parar · U1.3 Reactivo: Girar.



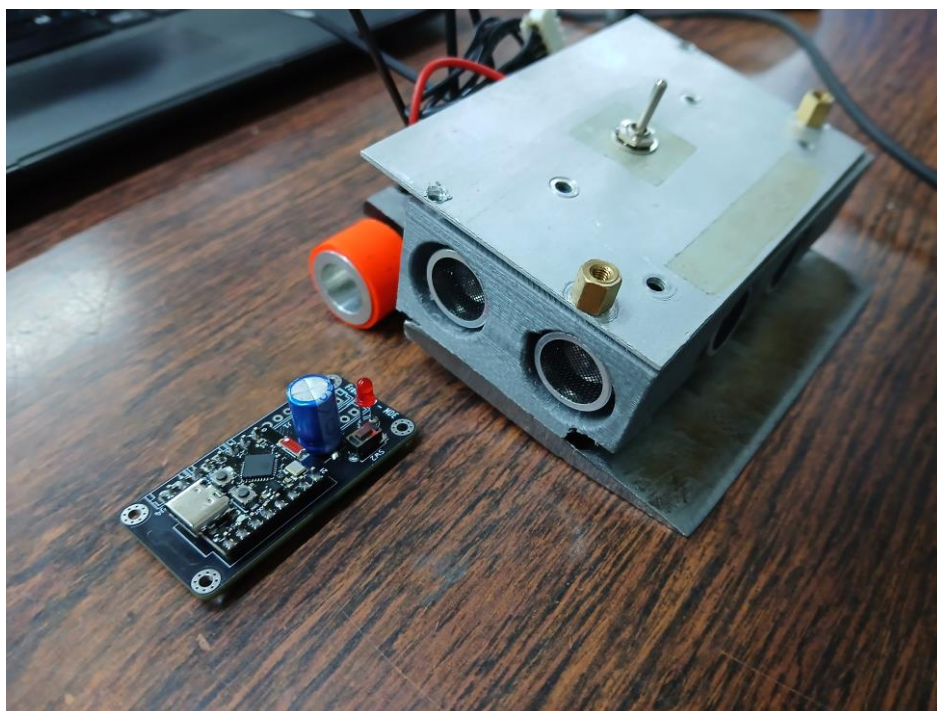
Unidad 2 — Programación Interactiva (3 desafíos)

Código estructurado con funciones, temporización sin delay() usando millis(), y control proporcional. U2.1 Secuencia con Tiempos · U2.2 Sin delay() · U2.3 Seguir la Pared (Control P).



Unidad 3 — Sensado y Datos (6+ desafíos)

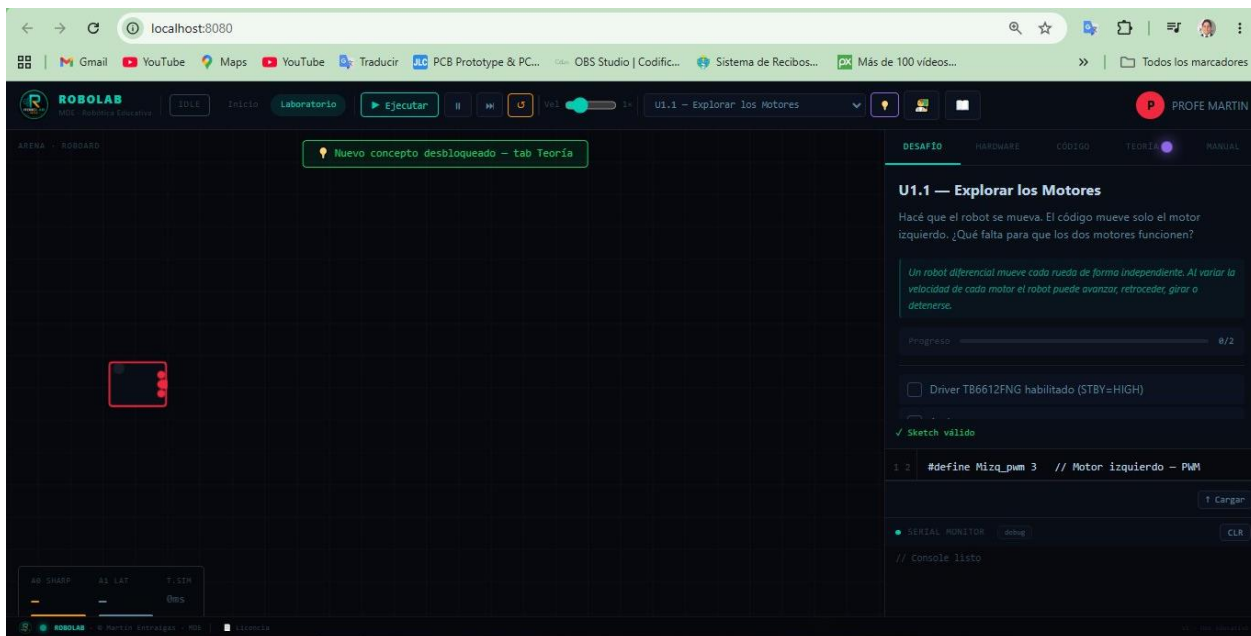
Sensores IR, minisumo autónomo, seguidor de línea con control PD. U3.1 Calibrar Sharp IR · U3.2 Minisumo: Detectar Borde · U3.3 Buscar y Atacar · U3.4 Seguidor de Línea · U3.5 Array QTR-8 · U3.6 Seguidor PD.



Robot FRANKY 4.0 en perspectiva — ESP32 C3, driver de motores, sensores ultrasónicos y pala delantera

Capítulo 3 — Recursos Disponibles en la Plataforma

Dentro de cada desafío de RoboLab, el estudiante cuenta con múltiples recursos integrados. No es necesario buscar información externa: todo lo necesario para resolver el desafío está disponible dentro de la plataforma.



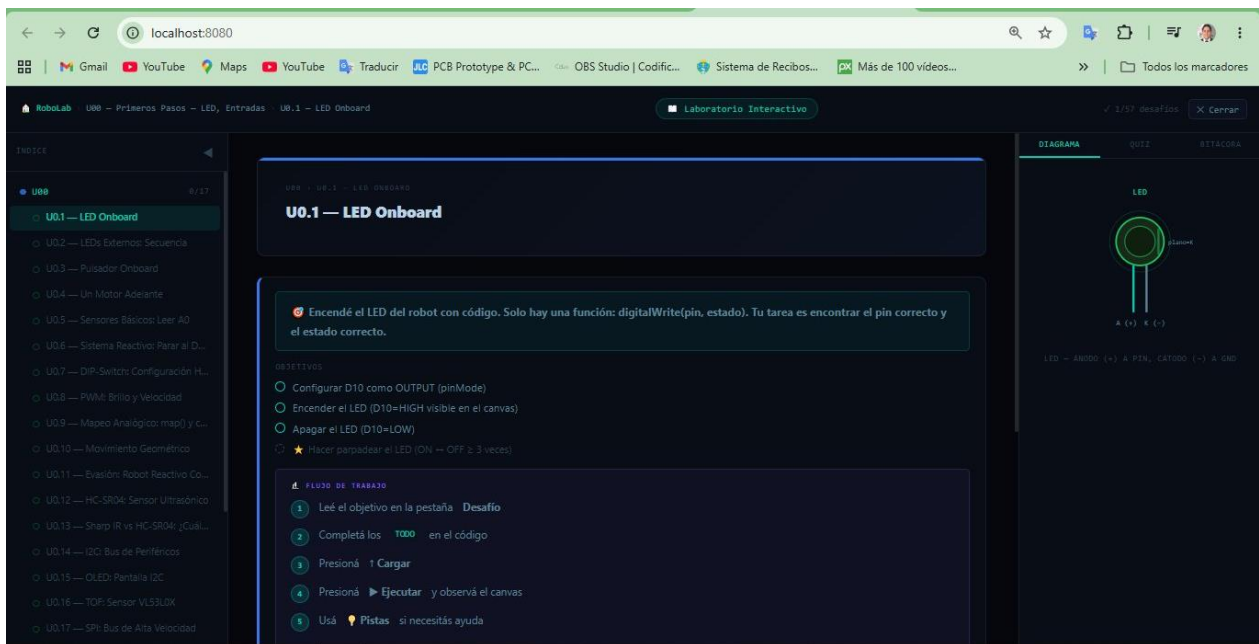
El Laboratorio Interactivo — simulador, editor de código, pestaña de desafío y monitor serial en una sola pantalla

Recursos del Laboratorio Interactivo

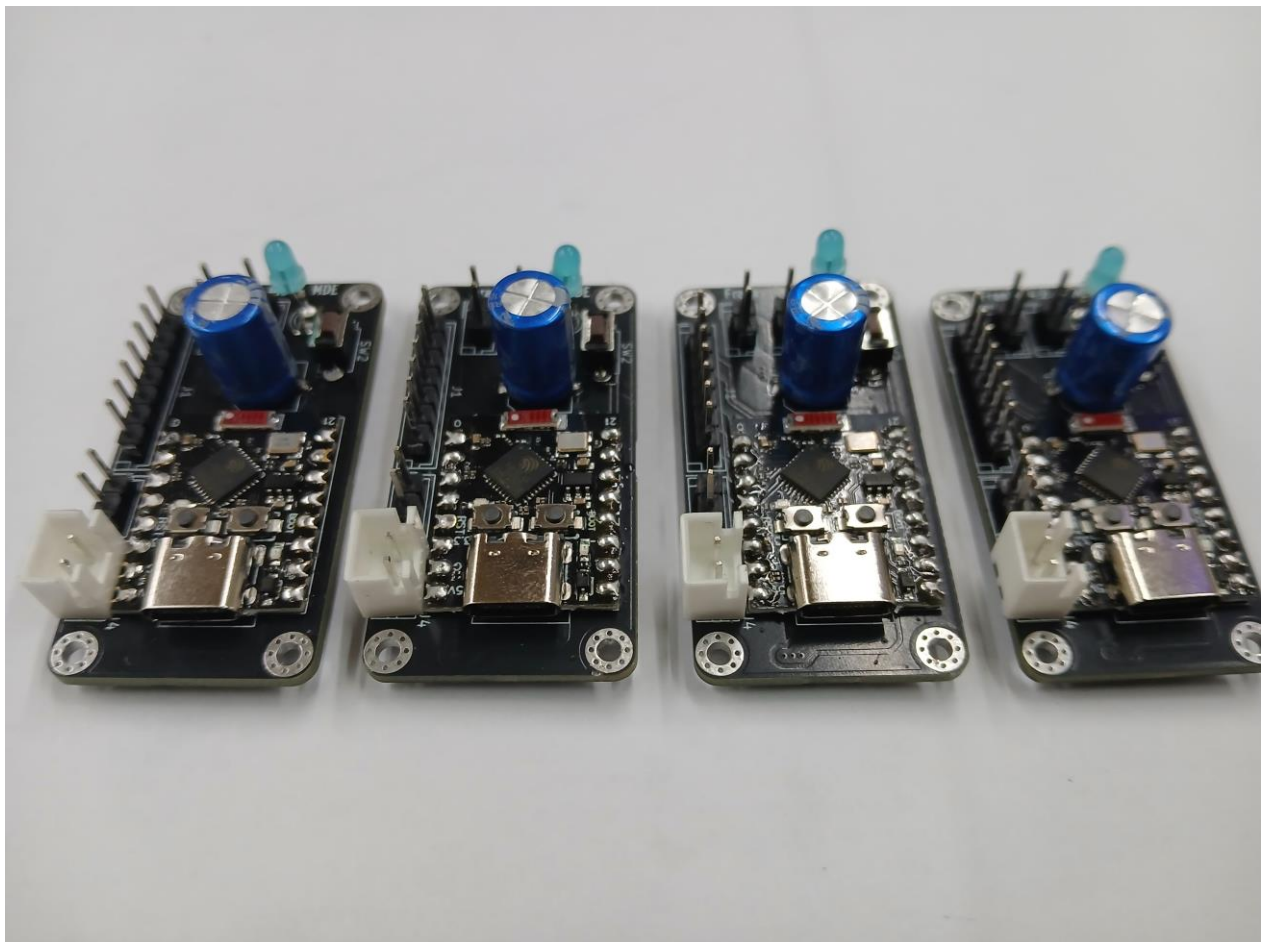
Recurso	Descripción
☐ Simulador (Canvas)	Área animada que representa el robot y su entorno. Al ejecutar el código, el robot responde en tiempo real: movimientos, LEDs, sensores y pantalla OLED.
☐ Editor de Código	Editor Arduino/C++ con resaltado de sintaxis. El estudiante completa los fragmentos marcados con TODO y hace clic en Cargar.
☐ Pestaña Desafío	Enunciado, objetivos del desafío y progreso. Los objetivos se marcan automáticamente al ser cumplidos.
☐ Pestaña Hardware	Diagrama del hardware involucrado: pines, conexiones, componentes. Muestra la relación entre el código y el circuito real.
☐ Pestaña Teoría	Explicación conceptual del tema. Se desbloquea progresivamente mientras el estudiante trabaja.
☐ Pestaña Manual	Referencia técnica: comandos, funciones y parámetros relevantes para el desafío.
☐ ? Pistas	Sistema de ayuda progresiva. El estudiante solicita pistas si no sabe cómo avanzar, sin revelar la solución completa.
☐ Bitácora	Cuaderno de registro personal para anotar observaciones, hipótesis y conclusiones.
☐ Quiz	Preguntas de comprensión para verificar el aprendizaje conceptual del desafío.

Serial Monitor

Monitor de salida. Muestra los mensajes `Serial.println()` del código, fundamental para depurar.



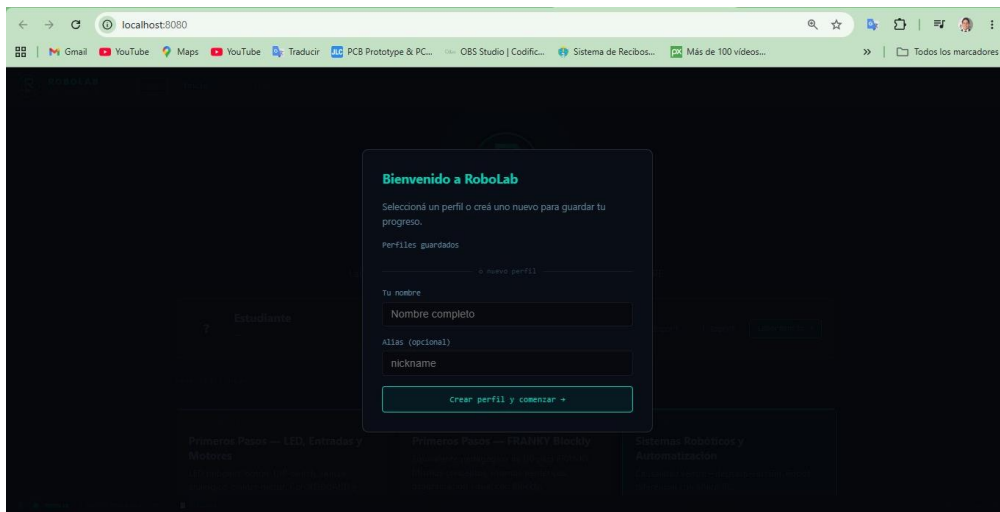
Cuaderno de Laboratorio - Vista de un desafío (U1.1 Explorar los Motores): enunciado, objetivos, código con TODOs y simulador del robot



Placas de control FRANKY 4.0 — ESP32-C3 con USB-C, capacitor de filtro y LED indicador (4 unidades en producción)

Capítulo 4 — Cómo Estudiar con RoboLab

RoboLab propone un flujo de trabajo concreto para cada desafío. Este flujo asegura que el estudiante comprenda, experimente y reflexione, sin saltarse pasos importantes.



Pantalla de bienvenida — crear o seleccionar un perfil para guardar el progreso

Flujo de Trabajo Recomendado por Desafío

1	Crear perfil	Al iniciar RoboLab, crear un perfil con nombre y alias. El progreso se guarda en el navegador.
2	Leer el enunciado	En la pestaña Desafío, leer qué debe hacer el robot y cuáles son los objetivos a cumplir.
3	Explorar el hardware	En la pestaña Hardware, ver qué pines y componentes intervienen en el desafío.
4	Revisar el código inicial	El editor trae un código parcial con espacios marcados TODO. Entender qué hace lo ya escrito.
5	Completar los TODOs	Escribir el código que falta. Si hay dudas, usar las Pistas o la pestaña Teoría.
6	Cargar y Ejecutar	Clic en Cargar para validar, luego en Ejecutar para ver el robot en el simulador.
7	Observar y verificar	Verificar que los objetivos se marquen como completados. Usar el Serial Monitor.
8	Registrar en Bitácora	Anotar qué se aprendió, qué fue difícil y qué se podría mejorar.
9	Completar el Quiz	Responder las preguntas conceptuales para consolidar el aprendizaje.



Consejo clave

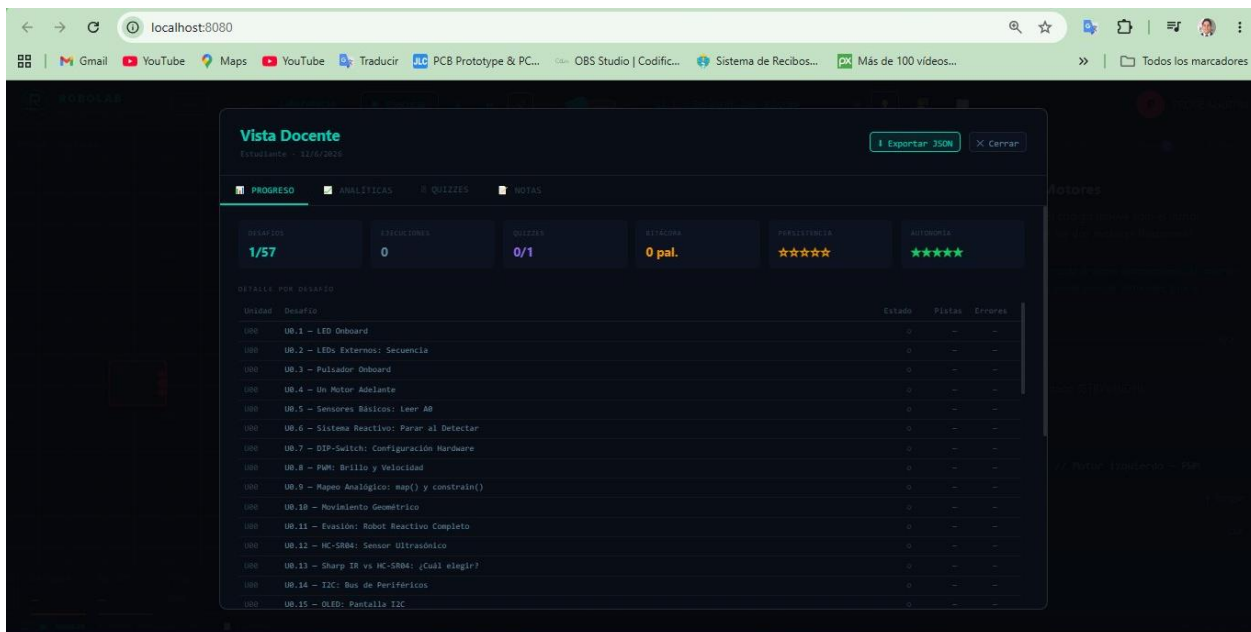
No avanzar al siguiente desafío hasta completar todos los objetivos requeridos del actual. La progresión de RoboLab está diseñada para que cada concepto nuevo se apoye en el anterior.

Seguimiento de Progreso

El perfil de usuario registra automáticamente: desafíos completados (sobre 57 totales), ejecuciones realizadas, quizzes respondidos, palabras escritas en bitácora, pistas utilizadas y tiempo de trabajo. Estos indicadores son visibles desde la Vista Docente.

Capítulo 5 — Cómo Enseñar con RoboLab

RoboLab es una herramienta flexible que se adapta a distintas modalidades de enseñanza. El docente puede elegir el enfoque según el contexto, los recursos disponibles y los objetivos pedagógicos.



Vista Docente — progreso detallado por estudiante: desafíos, ejecuciones, quizzes, bitácora, persistencia y autonomía

La Vista Docente

RoboLab incluye una herramienta especial para docentes: la Vista Docente. Permite ver el progreso detallado de cada estudiante, incluyendo desafíos completados, ejecuciones, quizzes, bitácora, uso de pistas y más.

- Ver cuántos de los 57 desafíos completó cada estudiante
- Identificar en qué desafío está bloqueado un estudiante
- Ver cuántas pistas usó (indicador de dificultad percibida)
- Revisar si completó los quizzes conceptuales
- Exportar el progreso en formato JSON para registros externos
- Acceder a analíticas del grupo para identificar desafíos problemáticos

Modalidades de Uso

☐ Clase Presencial

El docente presenta un desafío, explica el contexto y los estudiantes trabajan en sus PCs. Ideal para 10 a 30 estudiantes.



Sugerencia

Proyectar la pantalla del docente para mostrar el simulador en acción antes de que los estudiantes empiecen.

□ Taller / Maker Space

Los participantes eligen su propio ritmo. El facilitador está disponible para consultas. Funciona para grupos heterogéneos.



Sugerencia

Imprimir una guía rápida de inicio para que los participantes puedan comenzar sin esperar al facilitador.

□ Clase Invertida

Los estudiantes trabajan los desafíos en casa. El tiempo en clase se usa para resolver dudas y trabajar en proyectos grupales.



Sugerencia

Asignar desafíos específicos como tarea y revisar el progreso desde la Vista Docente antes de la clase.

□ Trabajo Colaborativo

Dos o más estudiantes trabajan juntos: uno escribe el código, el otro lee el enunciado y verifica los objetivos.



Sugerencia

Asignar roles: Programador y Observador. Rotar los roles en cada desafío para que ambos practiquen.

□ Aprendizaje Basado en Proyectos

Combinar los desafíos de U3 o E3–E5 en un proyecto integrador: minisumo o seguidor de línea funcional.



Sugerencia

Usar la Unidad 3 como base para organizar una mini-competencia de robots al final del curso.

Capítulo 6 — Secuencia de Aprendizaje Recomendada

La secuencia de RoboLab lleva al estudiante desde los conceptos más básicos hasta la programación avanzada de robots autónomos, preparándolo para abordar las plataformas físicas reales.

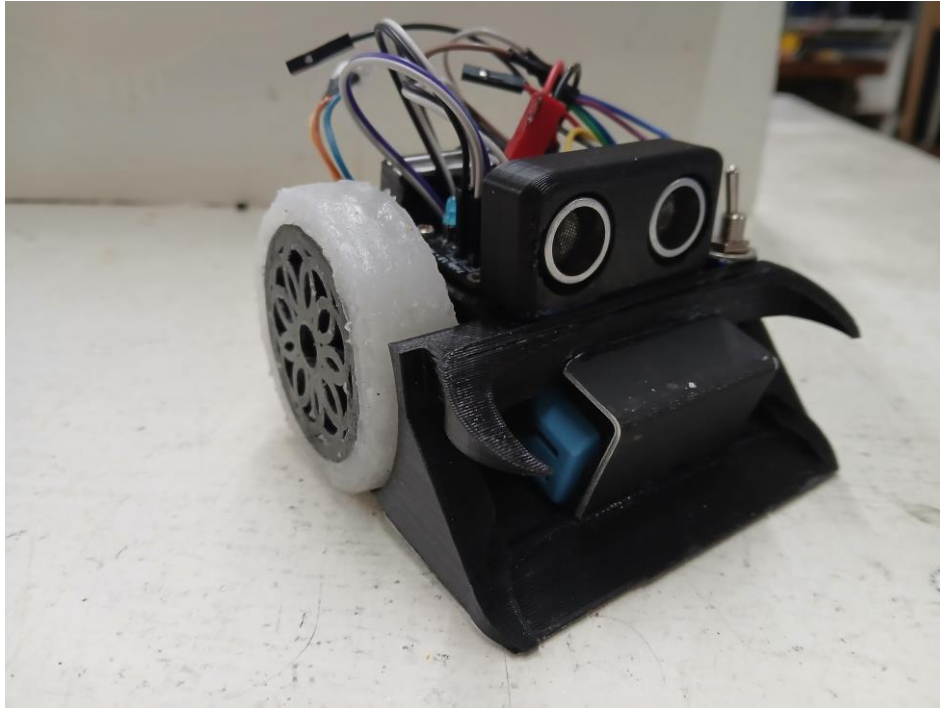
Diagrama de Progresión

Nivel	Unidad	Qué se aprende
□ INICIO	E0 — FRANKY Blockly (16 desafíos)	Programación visual con bloques. Sin necesidad de escribir código. Ideal para comenzar desde cero absoluto.
□ BASE	U0 — Primeros Pasos ROBOARD (17 desafíos)	Programación en C++/Arduino. LED, sensores, motores, buses I2C y SPI. Fundamentos del hardware.
□ MEDIO	U1 — Sistemas Robóticos (3 desafíos)	Robot diferencial reactivo. Sensor → decisión → acción.
□ MEDIO	U2 — Programación Interactiva (3 desafíos)	Funciones, millis(), control proporcional. Código más estructurado.
□ AVANZADO	U3 — Sensado y Datos (6+ desafíos)	Sensores IR, minisumo autónomo, seguidor de línea, control PD.
□ AVANZADO	E1-E5 — FRANKY Etapas (8 desafíos)	Control remoto, telemetría, modos autónomos, minisumo PD.
★ SIGUIENTE NIVEL	FRANKY 4.0 / ROBOARD 6.0 físicos	Al completar RoboLab, el estudiante está preparado para abordar las plataformas de hardware avanzadas.

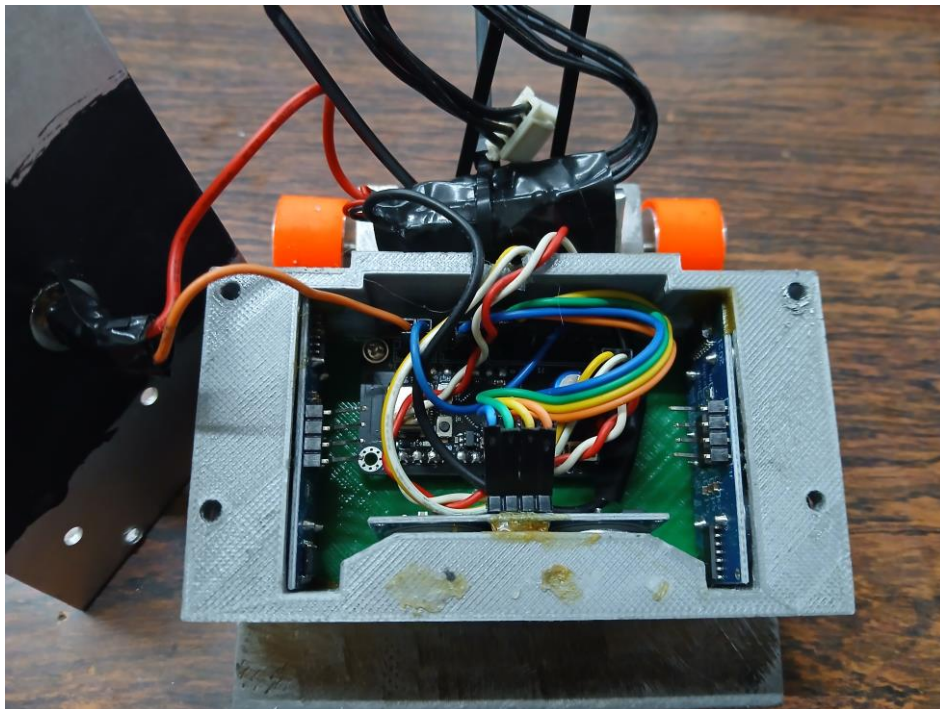
El camino continúa



Al completar RoboLab, el estudiante habrá adquirido los fundamentos necesarios para abordar plataformas más complejas: FRANKY 4.0 y ROBOARD 6.0 en su versión avanzada. RoboLab es el punto de partida de un ecosistema educativo completo.



Robot FRANKY-KID — chasis impreso en 3D con placa de control, sensores ultrasónicos y ruedas



Interior de un robot FRANKY 4.0 — placa principal, sensores y motores con conexiones de colores identificados

Capítulo 7 — Recomendaciones

Para Docentes

- Comenzar siempre con U0.1 o E0.1 para calibrar el nivel del grupo, incluso si creen saber programar.
- No forzar el ritmo: algunos estudiantes tardan 10 minutos en U0.1, otros 30 minutos. Ambos ritmos son válidos.
- Usar la Vista Docente regularmente para identificar quién necesita ayuda antes de que el estudiante la pida.
- Planificar unidades completas, no desafíos sueltos. Cada unidad tiene coherencia interna.
- Integrar la Bitácora como parte de la evaluación: refleja el proceso, no solo el resultado.
- Combinar RoboLab con el hardware real (ROBOARD o FRANKY) cuando sea posible para enriquecer la experiencia.

18

Para Estudiantes

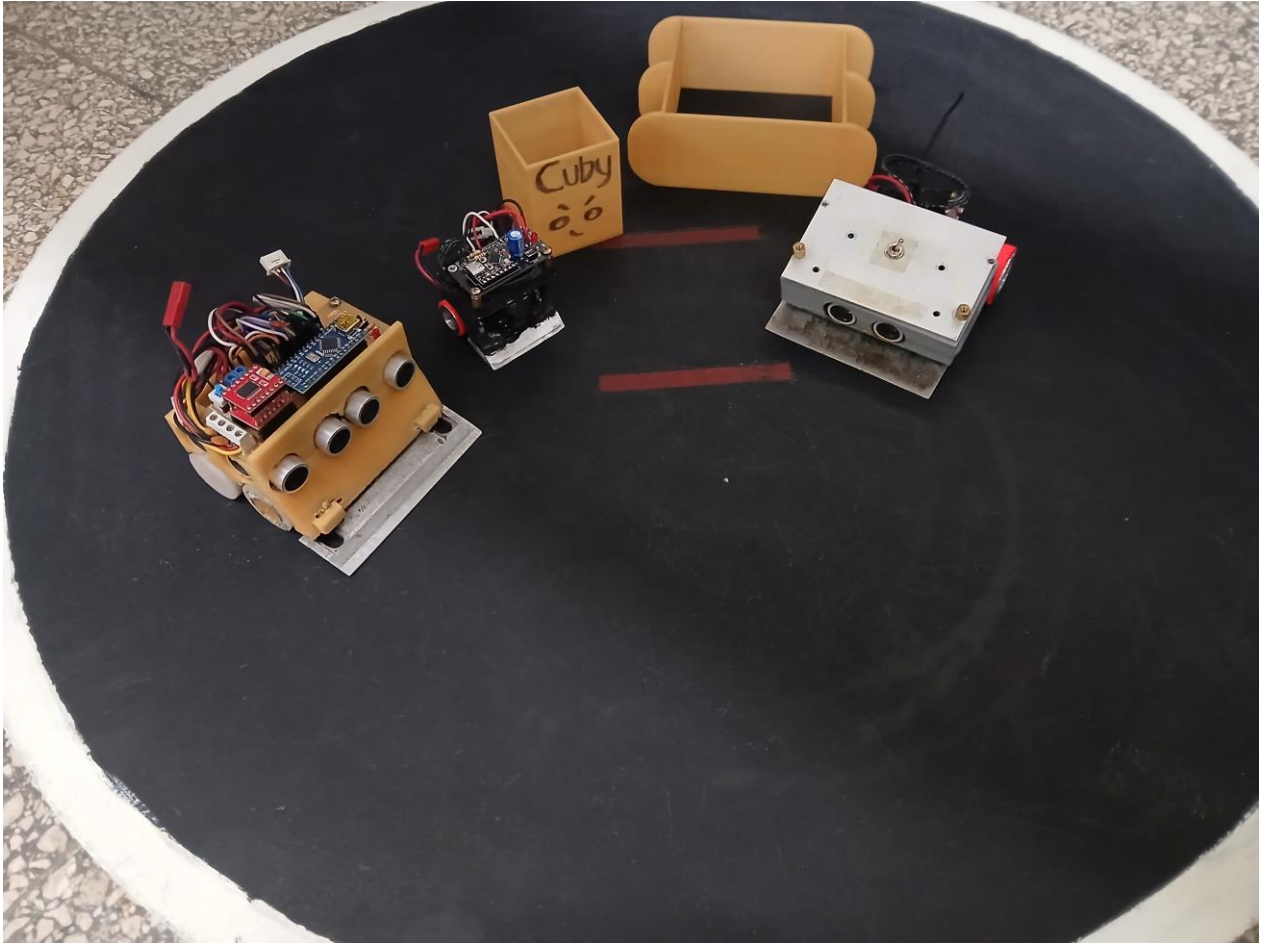
- Leer siempre el enunciado completo antes de escribir código.
- Usar las Pistas solo cuando realmente no se sabe cómo continuar, no como primer recurso.
- Anotar en la Bitácora aunque sea una sola línea por desafío.
- No copiar soluciones de internet: en el simulador, lo que funciona en otro contexto puede no funcionar aquí.
- Si el robot no se mueve como se esperaba, revisar el Serial Monitor: los mensajes explican qué está pasando.
- Celebrar los pequeños logros: cuando el LED parpadea por primera vez, es un hito real de aprendizaje.

Para Talleristas

- Definir un objetivo concreto para el taller: 'Al finalizar, habrán programado un robot reactivo'.
- Usar U0.1 a U0.6 para un taller de 3 horas con principiantes.
- Usar U0.1 a U0.11 para un taller de un día completo.
- Preparar el ambiente con Python instalado en las PCs o usar el modo LAN con un servidor central.
- Tener impresa una guía rápida de inicio por si hay problemas técnicos al comienzo.

Para Aprendizaje Autónomo

- Seguir la secuencia recomendada: E0 → U0 → U1 → U2 → U3.
- Dedicar al menos 30 minutos continuos por sesión para entrar en ritmo.
- Si un desafío tarda más de 20 minutos sin avanzar, usar Pistas o la pestaña Teoría.
- Combinar RoboLab con videos sobre Arduino o ESP32 en YouTube para complementar la teoría.
- Al completar una unidad completa, intentar reproducir lo aprendido en hardware real si está disponible.



Múltiples robots compitiendo en arena de minisumo — el objetivo final después de completar la Unidad 3

Capítulo 8 — Preguntas Frecuentes

? ¿Necesito un robot físico para usar RoboLab?

No. RoboLab funciona completamente con el simulador integrado. Podés programar, ejecutar y ver los resultados sin tener ningún hardware. El robot físico es opcional y enriquece la experiencia, pero no es requerido.

? ¿Qué necesito instalar en mi PC?

Solo Python 3 (gratuito en python.org) y un navegador web moderno como Chrome o Firefox. No se necesita instalar RoboLab: basta con copiar la carpeta del proyecto y ejecutar el archivo INICIAR_ROBOLAB.bat con doble clic.

? ¿RoboLab funciona sin conexión a internet?

Sí. RoboLab funciona completamente offline una vez que tenés la carpeta del proyecto. Todos los recursos — simulador, curriculum, teoría, quizzes — están incluidos localmente.

? ¿Puedo usar RoboLab con varios estudiantes al mismo tiempo?

Sí. Usando INICIAR_ROBOLAB_LAN.bat en una PC que actúe como servidor, todos los estudiantes de la red pueden conectarse desde sus navegadores escribiendo la IP del servidor. No necesitan instalar nada.

? ¿El progreso de los estudiantes se guarda?

El progreso se guarda en el navegador del equipo donde trabaja el estudiante (localStorage). Si cambia de PC, el progreso no se transfiere automáticamente, aunque puede exportarse manualmente en formato JSON desde la Vista Docente.

? ¿Por qué U0 y E0 tienen los mismos temas?

U0 usa programación en C++ (Arduino) para ROBOARD 6.0. E0 usa programación visual con Blockly para FRANKY 4.0. Los conceptos son idénticos, pero el lenguaje de programación y el hardware son diferentes.

? ¿Qué es el estado IDLE que aparece en la barra superior?

IDLE indica que el simulador está detenido y esperando. Cuando el robot está en ejecución, el estado cambia a RUNNING. Si el código tiene errores, el simulador no ejecuta y muestra el error en el editor.

? ¿Cómo sé si mi código es correcto?

Los objetivos del desafío se marcan automáticamente en verde cuando son cumplidos. El Serial Monitor muestra los mensajes de tu código. Si todos los objetivos requeridos están completos, el desafío está resuelto.

? ¿Qué son FRANKY 4.0 y ROBOARD 6.0?

Son las plataformas de hardware robótico compatibles con RoboLab. ROBOARD 6.0 está basada en Arduino Nano y FRANKY 4.0 en ESP32-C3. RoboLab simula ambas y permite subir el código al robot real cuando el estudiante está listo.

? ¿Dónde está la guía de instalación para docentes?

El archivo GUIA_DOCENTE.md incluido en la carpeta de RoboLab describe en detalle cómo distribuir y configurar la plataforma en laboratorios escolares, redes LAN, pendrives USB y más.

Capítulo 9 — Continuar Aprendiendo

RoboLab es el punto de partida de un ecosistema educativo más amplio. Al completar sus unidades, el estudiante estará preparado para abordar plataformas robóticas avanzadas que expanden lo aprendido hacia proyectos de mayor complejidad y hardware más sofisticado.

FRANKY 4.0 — Plataforma Educativa para Robótica Real

22



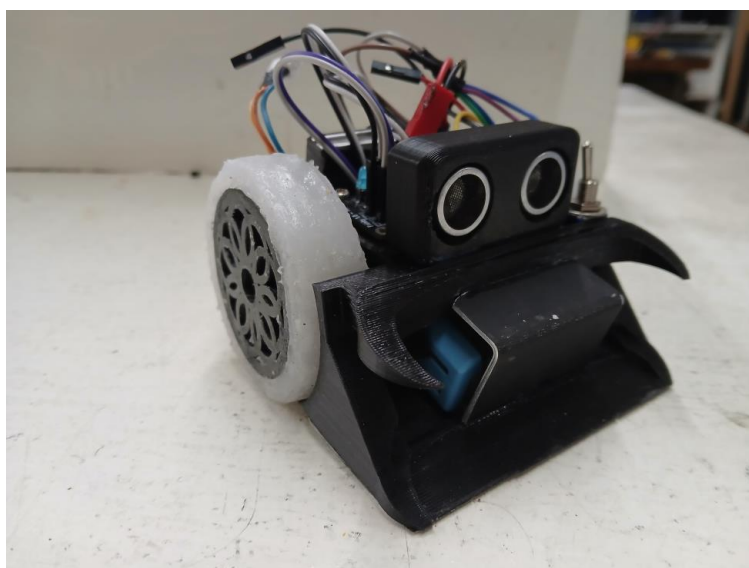
FRANKY 4.0 — Plataforma educativa para robótica real

FRANKY 4.0 es la plataforma avanzada para quienes completaron E0 y las etapas E1–E5 en RoboLab. Representa el siguiente nivel del ecosistema: más capacidades, mayor conectividad y proyectos más complejos. El trabajo con FRANKY en RoboLab es exactamente la preparación para usar la plataforma física real.

¿Cuándo pasar a FRANKY 4.0?



Cuando el estudiante haya completado E0 (Primeros Pasos Blockly) y E1 a E5 (Etapas FRANKY) en RoboLab. Al llegar a este punto, el código que programó en el simulador funciona directamente en el hardware real.



Prototipo de robot compatible con FRANKY 4.0 — construcción impresa en 3D

ROBOARD 6.0 — Plataforma Educativa para Robótica de Competencia



23

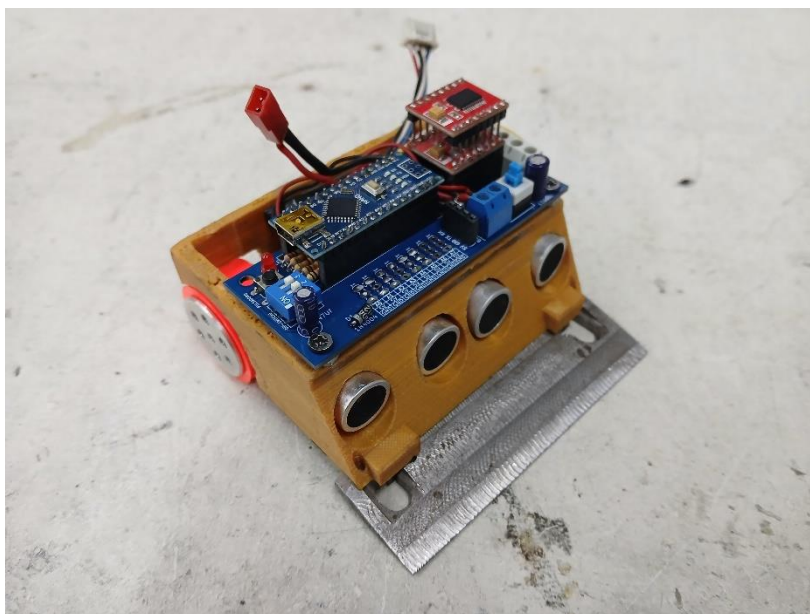
ROBOARD 6.0 — Plataforma educativa para robótica de competencia

ROBOARD 6.0 es la plataforma de referencia para robótica educativa y de competencia. Quienes completen U0, U1, U2 y U3 en RoboLab habrán desarrollado exactamente las habilidades necesarias para programar un robot ROBOARD real y participar en competencias de minisumo o seguidor de línea.

¿Cuándo pasar a ROBOARD 6.0?



Al completar U0, U1 y U2 de RoboLab. La Unidad 3 (minisumo y seguidor de línea) es la preparación directa para la competencia con hardware real. Al llegar a U3, el robot simulado se comporta igual que el robot físico.



Robot ROBOARD 6.0 listo para competencia — Arduino Nano + driver de motores + sensores ultrasónicos frontales

El Ecosistema Completo

RoboLab MDE	FRANKY 4.0	ROBOARD 6.0
Etapa formativa. Simulador + hardware real. Sin requisitos previos. ☐ PUNTO DE INICIO	Plataforma avanzada para robótica real. Requiere E0–E5 en RoboLab. ☐ NIVEL AVANZADO	Plataforma de competencia. Requiere U0–U3 en RoboLab. ☐ COMPETENCIA



Robots reales compitiendo en arena — el resultado de completar el ecosistema educativo RoboLab → ROBOARD 6.0 y FRANKY 4.0

Anexos

A. Glosario

Término	Definición
analogRead()	Función Arduino que lee un valor analógico (0–1023) de un pin analógico. Usada con sensores de distancia y luz.
Arduino / C++	Lenguaje de programación usado en U0–U3. Basado en C++, diseñado para microcontroladores de bajo costo.
Blockly	Entorno de programación visual por bloques. Usado en E0–E5 (FRANKY). No requiere escribir código.
Canvas	Área visual del simulador donde se representa el robot y su entorno en tiempo real.
Control P / PD	Algoritmos de control automático. P = proporcional. PD = proporcional + derivativo. Usados en seguidor de línea.
delay()	Función Arduino que pausa el programa por N milisegundos. Bloquea el robot. Se reemplaza por millis() en U2.2.
DIP-Switch	Interruptores de hardware que permiten configurar el robot físicamente sin programar.
digitalWrite()	Función Arduino para poner un pin en HIGH o LOW. Usada para encender LEDs y controlar motores.
ESP32-C3	Microcontrolador de FRANKY 4.0. Incluye WiFi y Bluetooth integrados. Más potente que Arduino Nano.
HC-SR04	Sensor ultrasónico que mide distancias emitiendo pulsos de sonido y midiendo el eco.
I2C	Bus de comunicación serial que conecta múltiples periféricos (OLED, sensores) con solo 2 cables.
millis()	Función Arduino que devuelve el tiempo en ms desde el inicio. Permite temporización sin bloquear el robot.
Minisumo	Modalidad de competencia robótica donde un robot intenta empujar al oponente fuera de una arena circular.
OLED	Tipo de pantalla pequeña usada para mostrar datos del robot (distancia, velocidad, estado).
PWM	Modulación por Ancho de Pulso. Simula una señal analógica con señales digitales. Controla velocidad y brillo.
Robot diferencial	Robot con dos ruedas motrices independientes. Gira variando la velocidad relativa de cada rueda.
Serial Monitor	Herramienta que muestra los mensajes enviados con Serial.println(). Fundamental para depurar código.
Sharp IR	Sensor de distancia infrarrojo analógico. Más económico que el ultrasónico, con rango de 10–80 cm.
SPI	Bus de comunicación de alta velocidad. Más rápido que I2C, usado para sensores de alta frecuencia.

TB6612FNG	Driver de motores integrado en ROBOARD 6.0. Controla dos motores DC con señales PWM y dirección.
TODO	Marcador en el código que indica dónde el estudiante debe escribir su solución.
TOF / VL53L0X	Sensor de distancia por Tiempo de Vuelo. Mide en milímetros con alta precisión usando láser infrarrojo.

B. Requisitos del Sistema

Componente	Mínimo	Recomendado
Sistema Operativo	Windows 7	Windows 10 / 11
RAM	2 GB	4 GB o más
Python	Python 3.6+	Python 3.10+
Navegador	Chrome 70 / Firefox 90	Chrome 110+ o Edge reciente
Pantalla	1280×720	1366×768 o mayor
Espacio en disco	15 MB	—
Conexión a internet	No requerida	Solo para actualizaciones

C. Créditos

RoboLab MDE fue desarrollado como plataforma educativa para la enseñanza progresiva de robótica, electrónica y programación para educación secundaria técnica y nivel superior.

Plataforma: RoboLab MDE — Aprender · Programar · Crear

Plataformas de hardware: ROBOARD 6.0 · FRANKY 4.0

Tecnología: HTML5 · JavaScript · Blockly · Python (servidor) · Simulador integrado

Versión del manual: 1.0 — Junio 2026



APRENDER · PROGRAMAR · CREAR